

REPORTE DE CALIDAD

Frontend SaaS POS System

Análisis tipo SonarQube de código fuente
Vue 3 + TypeScript + Vuetify 3



Fecha: 8 de Marzo de 2026
Herramienta: Claude Code Analysis Engine
Rama analizada: dev

RESUMEN EJECUTIVO

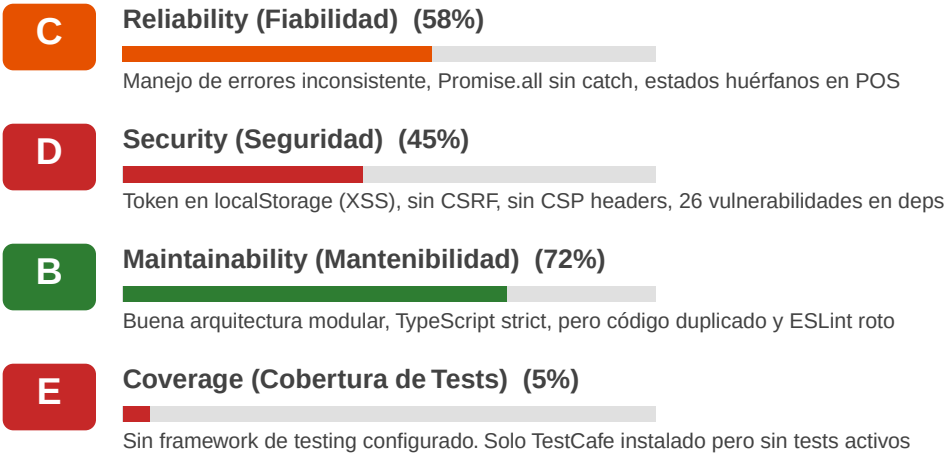
Se realizó un análisis exhaustivo del código fuente del frontend SaaS POS, evaluando seguridad, calidad de código, mantenibilidad, cobertura de pruebas, duplicación, manejo de errores, y adherencia a buenas prácticas. El análisis cubre los 14 módulos de la aplicación, la infraestructura core, configuración del proyecto y dependencias.

El sistema presenta una arquitectura modular bien diseñada con buena separación de responsabilidades, pero tiene deficiencias críticas en seguridad, manejo de errores, validaciones y configuración que deben resolverse antes de salir a producción.

Métricas Generales del Proyecto

Métrica	Valor
Archivos TypeScript/Vue analizados	~100+
Módulos funcionales	12 activos + 2 stubs
Vulnerabilidades en dependencias	26 (16 high, 9 moderate, 1 low)
Issues críticos encontrados	18
Issues de alta prioridad	35
Issues de media prioridad	42
Issues menores	25
Total de issues	120

Calificaciones por Dimensión (Estilo SonarQube)



C**Duplications (Duplicación) (55%)**

Servicios fetch idénticos, lowStock duplicado, formatDateTime repetido, validaciones inline

D**Security Hotspots (40%)**

15 hotspots: .env con keys, reCAPTCHA sin error handling, role bypass, `as any` casts

ANÁLISIS DETALLADO POR ÁREA

1. SEGURIDAD

1.1 Vulnerabilidades Críticas

- Token JWT almacenado en localStorage — vulnerable a XSS (auth.store.ts:20,31)
- Sin protección CSRF — no se envían tokens X-CSRF-Token (client.ts)
- Sin Content Security Policy (CSP) — permite inyección de scripts (index.html)
- 26 vulnerabilidades en dependencias npm (16 high severity)
- reCAPTCHA sin manejo de errores — login falla silenciosamente (recaptcha.ts:1-10)
- Clave reCAPTCHA en .env committed al repositorio

1.2 Autenticación y Autorización

- Sin token refresh — token expira sin renovación automática (auth.store.ts)
- Logout no invalida token en backend — token sigue válido server-side
- localStorage.clear() borra TODO, no solo datos de auth
- Role check case-sensitive puede causar bypass (guards.ts:17-18)
- UserRole treats empty role as admin — peligroso (ProductListView.vue:141)
- Sin verificación de token al iniciar app — acepta tokens expirados

1.3 Validación de Datos

- Email regex débil: /.+@.+\./ acepta formatos inválidos (LoginView.vue:75)
- Sin validación de formato NIT colombiano en Settings y Suppliers
- Sin validación de respuesta del API — acepta cualquier estructura
- Tipo `as any` usado en 15+ lugares — anula TypeScript strict mode

2. FIABILIDAD (Reliability)

2.1 Manejo de Errores

- POS Store: loadOpenOrders() sin try-catch — loading queda en true si falla (pos.store.ts:58-71)
- POS Store: createOrder() sin error handling — fallo silencioso (pos.store.ts:73-85)
- Promise.all() sin manejo individual — un fallo bloquea todo (PosView.vue:257-272)
- InvoiceListView Promise.all sin catch (InvoiceListView.vue:285)
- Reportes: error se muestra como "sin datos" — ambiguo (DailyReportView.vue:221)
- Cash Register: closeRegister retorna silenciosamente si null (store:27-30)

2.2 Condiciones de Carrera (Race Conditions)

- CRÍTICO: closeOrder captura estado antes de confirmación API (pos.store.ts:114-138)
- addProduct + removeItem pueden ejecutarse simultáneamente sin lock
- cashRegisterOpen puede retornar stale durante carga inicial
- Doble submit posible en PosPaymentDialog — cancel disponible durante processing
- Inventory reload N+1: cada add/remove hace 2 full reloads

2.3 Estados Inconsistentes

- activeOrderId se puede null mientras order aún existe en openOrders
- hasUsedPos se setea antes de confirmar que loadOpenOrders tuvo éxito
- Multi-order tabs: orderTabs computed existe pero nunca se usa en UI
- Success notification antes de completar reloads cascados

3. MANTENIBILIDAD (Maintainability)

3.1 Fortalezas

- Arquitectura modular bien definida — cada módulo con types/services/views/routes
- TypeScript strict mode habilitado con path aliases (@/, @core/, @modules/)
- Composable useCrud genérico para vistas CRUD estándar
- Pinia stores por módulo con buena separación de estado
- Layouts dinámicos (Default, Auth, Fullscreen)
- Vue 3 Composition API con <script setup lang="ts"> consistente

3.2 Debilidades

- ESLint ROTO: 48+ errores de parsing TypeScript — @babel/eslint-parser no soporta TS
- Falta @typescript-eslint/parser y plugin — sin reglas TS
- lintOnSave: false — errores no se detectan en desarrollo
- Código duplicado: servicios fetch idénticos en 4+ archivos
- lowStockProducts computed duplicado en SupplierListView y ProductListView
- formatDateTime duplicado en PurchaseOrderDetailView y DailyReportView
- Payment method labels hardcoded en PosView y PosPaymentDialog
- Validación inline repetida en cada componente CRUD
- data.data ?? data pattern fragil repetido en 10+ servicios

3.3 Deuda Técnica

- Servicios solo tienen fetch — falta create/update/delete en capa de servicio
- useCrud hace llamadas API directas sin pasar por servicios
- Sin paginación server-side — todo se carga en memoria del cliente
- Sin debounce en búsquedas — computa en cada keystroke
- Módulos business/ y subscription/ son stubs sin funcionalidad
- TestCafe instalado (~1GB) pero sin tests configurados

4. COBERTURA DE PRUEBAS (Test Coverage)

ALERTA CRÍTICA: Sin Framework de Testing Configurado

El proyecto no tiene tests unitarios, de integración ni end-to-end funcionando. TestCafe 3.7.4 está instalado como dependencia pero no hay archivos de test activos ni scripts de ejecución configurados. La carpeta testsprite_tests/ existe pero no está integrada al pipeline.

Impacto en producción:

- Sin tests, cada deploy es un riesgo — regresiones no se detectan
- Flujos críticos (POS, pagos, inventario) no tienen cobertura automatizada
- Refactoring es peligroso sin red de seguridad de tests
- Imposible hacer CI/CD confiable sin test suite

Tests mínimos requeridos antes de producción:

- Auth: login, logout, token refresh, role guards
- POS: crear orden, agregar producto, cobrar, cancelar
- Inventario: CRUD productos, movimientos, stock bajo
- Caja: abrir, cerrar, validación de montos
- Facturas: crear, cancelar, validación de stock

5. ESTADO POR MÓDULO

Módulo	Estado	Rating	Issues	Observaciones
Auth (Login)	FUNCIONAL	C	14	Falta: password reset, MFA, token refresh, validación respuesta API
Dashboard	FUNCIONAL	B	4	Básico pero funcional. Falta: gráficos, acciones directas, loading state visible
Catálogo (Cat/Sup/Cli)	FUNCIONAL	C+	18	CRUD funciona via useCrud. Falta: validaciones completas, paginación server-side
Inventario	FUNCIONAL	C+	12	Productos + movimientos OK. Falta: validaciones precio, paginación, audit trail
POS	CON RIESGOS	D+	25	CRÍTICO: Race conditions, sin error handling, N+1 API calls, tabs incompletas
Caja Registradora	BUENO	B	5	Bien implementado. Falta: difference alert colors, validación de montos
Compras	BUENO	B+	4	Excelente flujo de recepción con checklist. Password verification OK
Facturas	FUNCIONAL	C+	8	Validación stock OK. Falta: function name mismatch verifyAdminPassword
Admin/Usuarios	BUENO	B	5	Password strength indicator OK. Falta: duplicate email check, audit log
Reportes	BÁSICO	C	7	Solo reporte diario. Falta: rango fechas, export PDF/Excel, gráficos
Settings	FUNCIONAL	C+	6	Logo upload con memory leak. Falta: perfil usuario, 2FA, preferencias
Business	STUB	E	0	Solo servicio con fetchBusiness(). Sin vistas ni rutas
Subscription	STUB	E	0	Solo servicio con fetchSubscription(). Sin vistas ni rutas
Core Infrastructure	FUNCIONAL	C	16	API client OK. Router guards incompletos. Inactivity timer con memory leak potential

6. VULNERABILIDADES EN DEPENDENCIAS

Paquete	Severidad	Descripción
SVGO 2.1.0-2.8.0	HIGH	DoS via entity expansion (Billion Laughs)
Underscore <=1.13.7	HIGH	Unlimited recursion DoS en _.flatten/_.isEqual
Cross-spawn <6.0.6	HIGH	ReDoS en yorkie dependency
serialize-javascript	HIGH	Vulnerability en webpack plugins
webpack-dev-server <=5.2.0	HIGH	Source code theft via malicious sites
httpntlm (TestCafe chain)	HIGH	Vulnerable underscore dependency
@vue/cli-plugin-* 5.0.x	HIGH	Multiple transitive vulnerabilities

Total: 26 vulnerabilidades (16 HIGH, 9 MODERATE, 1 LOW). La mayoría provienen de la cadena de TestCafe y @vue/cli-service. Se recomienda migrar a Vite y eliminar TestCafe.

SCORE DE PRODUCCIÓN

Desglose del Score (100 pts máximo)

Categoría	Máx	Score	Detalle
Funcionalidad Core	25	20	12/14 módulos funcionales, flujos principales OK
Seguridad	20	8	Token en localStorage, sin CSP, 26 vulns en deps, sin CSRF
Manejo de Errores	15	8	Inconsistente, race conditions en POS, silent failures
Cobertura de Tests	15	1	Sin tests configurados ni ejecutándose
Calidad de Código	10	7	TypeScript strict, buena estructura, pero ESLint roto
Mantenibilidad	10	7	Modular, composables, pero código duplicado significativo
Configuración/DevOps	5	2	Sin .env.production, lintOnSave false, sin CI/CD visible
TOTAL	100	53 / 100	53% — Rating C

Interpretación del Score

A (90-100%): Listo para producción. Seguro, testeado, mantenible.
B (75-89%): Casi listo. Issues menores que no bloquean deploy.
! C (60-74%): Necesita trabajo. Issues significativos que afectan calidad.
D (40-59%): No recomendado. Riesgos altos de seguridad y estabilidad.
E (0-39%): No apto. Requiere rediseño o reescritura significativa.

7. BLOCKERS PARA PRODUCCIÓN

1. Corregir 26 vulnerabilidades en dependencias (npm audit fix + eliminar TestCafe)
2. Implementar manejo de errores consistente en POS Store (loadOpenOrders, createOrder)
3. Arreglar race conditions en closeOrder — no capturar estado antes de API response
4. Agregar CSP headers y mover token a HttpOnly cookies (o al menos agregar CSP)
5. Configurar ESLint con @typescript-eslint/parser — actualmente 48 errores de parsing
6. Crear .env.production con URLs de producción y remover .env del repositorio
7. Implementar token refresh o validación al iniciar la app
8. Agregar tests mínimos para flujos críticos (auth, POS, pagos)
9. Arreglar bug verifyAdminPassword — nombre de función incorrecto en InvoiceService
10. Agregar validación de formularios antes de submit (form.validate() en LoginView)

8. PLAN DE ACCIÓN RECOMENDADO

Fase 1: Crítico — Antes de Producción (1-2 semanas)

- Ejecutar `npm audit fix --force` y eliminar TestCafe si no se usa
- Arreglar ESLint: instalar `@typescript-eslint/parser` + plugin, habilitar `lintOnSave`
- Implementar error handling en POS store (try-catch en todas las funciones async)
- Arreglar race condition en `closeOrder` (mover state cleanup después de API success)
- Agregar prevención de doble-submit en `PosPaymentDialog`
- Crear `.env.production`, `.env.example`; remover `.env` y `.env.development` del git
- Agregar CSP meta tag en `index.html`
- Implementar logout con invalidación de token en backend (POST `/api/logout`)
- Arreglar memory leak en `SettingsView` (revoke `objectURL`)
- Arreglar bug `verifyAdminPassword` en `InvoiceService`

Fase 2: Alta Prioridad — Sprint siguiente (2-3 semanas)

- Implementar token refresh automático o validación al iniciar app
- Agregar tests unitarios: auth store, POS store, `useCrud` composable
- Agregar tests E2E: login, crear orden POS, cobrar, cerrar caja
- Implementar paginación server-side en catálogo e inventario
- Extraer código duplicado: servicios `fetch`, `lowStock`, `formatDateTime`
- Agregar validaciones completas en formularios (precios, NIT, documento)
- Implementar debounce en búsquedas de catálogo
- Completar multi-order tabs en POS (`orderTabs computed` ya existe)

Fase 3: Mejoras — Próximas iteraciones (1-2 meses)

- Migrar de Vue CLI (webpack) a Vite para builds más rápidos
- Implementar módulo Business completo (onboarding, invitaciones)
- Implementar módulo Subscription (planes, billing, upgrades)
- Agregar reportes avanzados: rango de fechas, gráficos, export PDF/Excel
- Implementar 2FA y perfil de usuario en Settings
- Agregar audit log en módulo Admin
- Implementar lazy loading de rutas para reducir bundle size
- Migrar token de `localStorage` a `HttpOnly cookies`

CONCLUSIÓN

El frontend del sistema SaaS POS tiene una base arquitectónica sólida con Vue 3, TypeScript strict mode, y una buena modularización por dominio de negocio. Los flujos principales (auth, catálogo, inventario, POS, caja, facturas, reportes) están implementados y funcionales.

Sin embargo, el sistema NO está listo para producción en su estado actual. Los principales bloqueantes son:

- Seguridad insuficiente (token XSS-vulnerable, 26 vulns en deps, sin CSP)
- Sin cobertura de tests (riesgo alto de regresiones)
- Race conditions y error handling deficiente en el módulo POS (el más crítico del negocio)
- ESLint completamente roto — no detecta errores de TypeScript

Con las correcciones de Fase 1 implementadas, el score subiría a ~75-80% (Rating B), suficiente para un MVP controlado con monitoreo activo. Las fases 2 y 3 llevarían el sistema a un nivel de producción robusto (Rating A).